

1. 公式推广

1.1 state value

1.1.1 定义

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \lambda(k) r_{t+k+1} | S_t = s]$$

1.1.2 推导

在经典情形下,

$$G_t = r_{t+1} + \lambda r_{t+2} + \lambda^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \lambda^k r_{t+k+1}$$

将 λ^k 替换为 $\lambda(k)$, 有:

$$G_t = \sum_{k=0}^{\infty} \lambda(k) r_{t+k+1}$$

在经典情形下, 有:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[r_{t+1} + \lambda V^{\pi}(S_{t+1}) | S_t = s]$$

这依赖于: $G_t = r_{t+1} + \lambda G_{t+1}$

将 λ^k 推广至 $\lambda(k)$, 有:

$$G_t = \lambda(0)r_{t+1} + \lambda(1)r_{t+2} + \lambda(2)r_{t+3} + \lambda(3)r_{t+4} + \dots$$

$$G_{t+1} = \lambda(0)r_{t+2} + \lambda(1)r_{t+3} + \lambda(2)r_{t+4} + \dots$$

观察可知, $r_{t+2}, r_{t+3}, r_{t+4}, \dots$ 的系数之比为:

$$\frac{\lambda(1)}{\lambda(0)}, \frac{\lambda(2)}{\lambda(1)}, \frac{\lambda(3)}{\lambda(2)}, \dots$$

全是变值, 无法找到一个值 β , 满足:

$$G_t = \lambda(0)r_{t+1} + \beta G_{t+1}, \quad \text{除非 } \lambda(k) = \lambda^k$$

这意味着，不存在：

$$V^\pi(s) = \mathbb{E}_\pi[\lambda(0)r_{t+1} + \beta V^\pi(S_{t+1}) \mid S_t = s]$$

1.1.3 Bellman 方程

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \lambda V^\pi(s')]$$

用 $\lambda(k)$ 替代 λ^k ，有：

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [\lambda(0)r + V_\beta^\pi(s')]$$

其中：

$$V_\beta^\pi(s') = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \lambda(k+1) r_{t+k+2} \mid S_{t+1} = s']$$

1.2 action value function

1.2.1 定义

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \lambda(k) r_{t+k+1} \mid S_t = s, A_t = a]$$

1.2.2 推导

$Q^\pi(s, a)$ 其实是限定 $A_t = a$ 时的 $V^\pi(s)$ ，有：

$$Q^\pi(s, \pi(s)) = V^\pi(s)$$

所以定义式的推导和 $V^\pi(s)$ 相同，将 λ^k 替换为 $\lambda(k)$ ，并且增加 $A_t = a$ 的条件。

同理可推得，经典情形下的：

$$Q^\pi(s, a) = \mathbb{E}_\pi[r_{t+1} + \lambda Q^\pi(S_{t+1}, \pi(S_{t+1})) \mid S_t = s, A_t = a]$$

在这里不一定成立。

1.2.3 Bellman 方程

$$Q^\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \lambda \sum_{a'} \pi(a' \mid s') Q^\pi(s', a') \right]$$

用 $\lambda(k)$ 替换 λ^k 后，为：

$$Q^\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[\lambda(0)r + \sum_{a'} \pi(a' \mid s') Q_\beta^\pi(s', a') \right]$$

其中：

$$Q_\beta^\pi(s', a') = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \lambda(k+1) r_{t+k+2} \mid S_{t+1} = s', A_{t+1} = a']$$

2. 算法训练复现

2.1 数据下载及处理

Thyme-SFT 数据集包括 single（单轮图片问答）、comp（无图片计算问答）和 2round（2 轮图片问答）三种数据，分别下载了 334、333 和 333 条数据，然后提取数据中的 image、question 和 response 中的 answer 部分组成 RL 问答格式数据集。

我将原始数据中的 base64 编码的 image 解码为 png 格式的图片并单独保存，然后把 question 中的图片路径替换为真实的本地路径。Single 和 2round 两种数据的 question 中没有介绍 sandbox 和代码执行，我仿照 comp 中的格式并结合具体的训练流程实现，为它们加上了合适的 prompt。

2.2 PPO 训练流程复现

2.2.1 实验环境：

A800*1+python 3.10+torch 2.5.1+cuda 12.4+transformers 5.5.4

2.2.2 实验设置

本次考核使用 Qwen-2.5-VL-3B-Instruct 作为 base model。各组件情况如下：

Policy: base model + Lora

Critic: base model + Lora + value_head

Reference: base model

RM: Qwen3.5-4B

使用 Lora 进行微调可以显著减少显存占用，并避免微调全体参数可能导致的灾难性遗忘问题。Value_head 是线性层，把 transformer blocks 最后一层的输出映射为期望 value。Reference 被用于施加 KL 惩罚，避免 Policy 偏离 base model 太远。考虑到某些问题的答案是描述性内容、模型可能在回答选项类问题时输出选项内容而不是选项本身等问题，我使用 Qwen3.5-4B（非思考模式）作为 RM。它被要求直接输出一个 0-10 的分数来衡量 Policy 的回答和 ground truth 之间的相似性，随后这个分数会被映射到 -1~1，作为 PPO 的最后一轮的 reward。

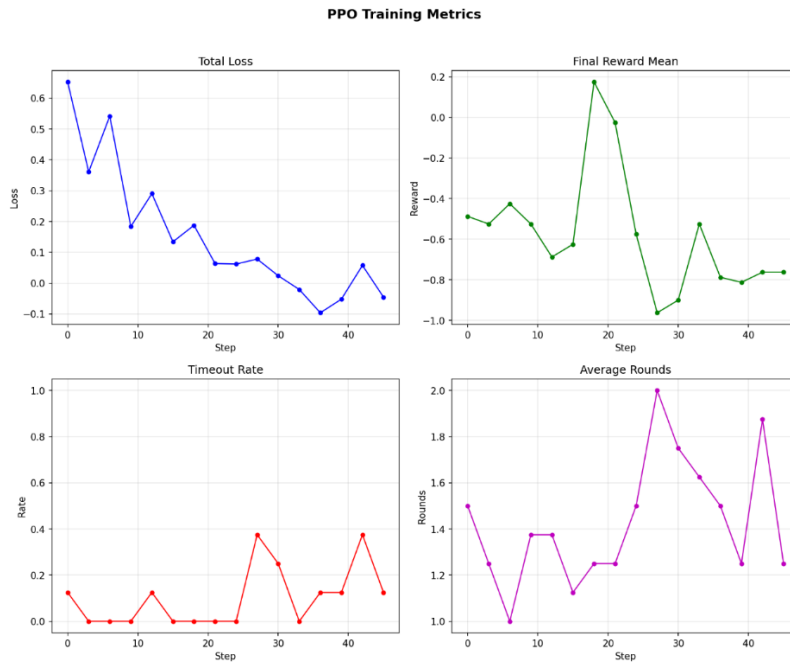
在 rollout 时，Policy 被限制为最多交互 3 轮。在中间轮次，系统会根据 Policy 输出的格式和代码执行情况给与较小的奖励或惩罚作为 reward。如果最终输出 answer，使用 RM 进行评分得到 final reward，然后反向计算每一轮的 cumulated reward；如果三轮结束都没有输出答案，给予超时惩罚作为 final reward。为了限制显存占用，原始数据中尺寸超过 256*256 的图片，会被缩放到 256*256；sandbox 返回的文本结果

（执行结果或报错信息）会被截断处理（图片会被完整传给模型），并且上下文只保留初始 message 和最近一轮的 message。为了避免 Policy 把不同样本处理后的图片都保存为“figure1”之类的名字导致覆盖问题，系统会为每一个样本单独创建一个临时文件夹，Policy 只能在这下面进行读写操作（此外，Policy 唯一被允许执行读操作的文件夹是存储 image 的文件夹）。

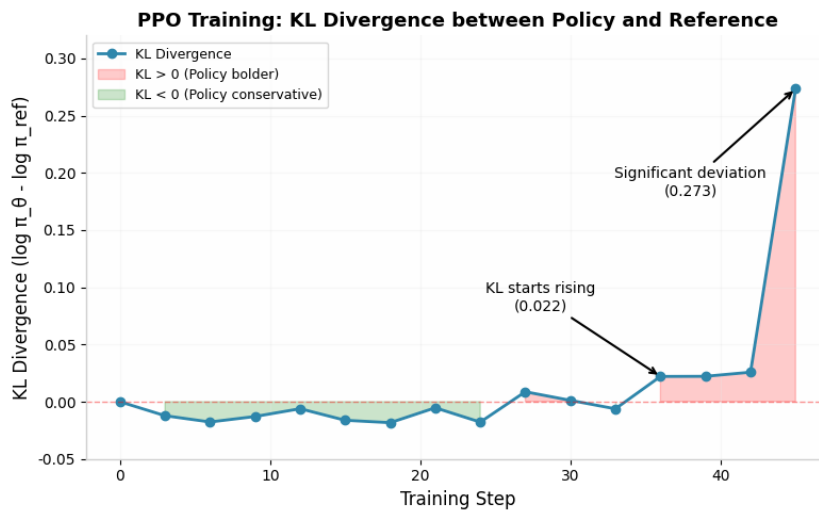
本次实验中，bs=8，ppo_epoch=3， $\epsilon=0.1$ ，timeout_penalty=-1.3。

2.2.3 实验结果

如下图所示，训练过程中，loss、final_reward、timeout_rate 和 average_rounds 四个指标变化如图所示。Loss 在训练过程中逐渐下降，说明训练的确起到了作用。final_reward 绝大部分时期都为负，这部分由于 RM 的评判比较严格。为了减少 RM 对显存的占用，它在非思考模式下运行，并且被要求直接输出 0-10 的分数。经过观察，我发现它倾向于输出接近 0 或者 10 的分数。如果 Policy 不足以完全正确回答某个问题，它的 final_reward 通常更接近 -1。



在训练过程前中期，KL 散度的绝对值保持在 0 附近。从第 36 step 开始，KL 散度开始增加，到第 45 step 时激增。实际上，在下一 step 时，训练因为 OOM 而终止了——这通常是因为存在样本跑满 3 轮并且生成了最大长度限制的 response，message 的长度激增而导致显存爆炸。考虑到 timeout_rate 在 36 step 之后显著高于训练前期，Policy 可能开始学会更加激进的进行分析或者使用代码。



下面展示了第 42 和 45 step 中轮数最长的样本的 final_response 的部分结果。第 42 step 时模型输出了 answer，不过没有按照格式输出会被判定为超时；第 45 step 时，模型在最后一轮试图执行代码。

Step 0042

Sample (R3): B. The mean trend of the data over the given range of γ

Step 0045

Sample (R3):

<think>

```
```python
```

```
print("output.jpg")
```

```
``` ...
```

2.3 GRPO 训练复现

2.3.1 实验设置

GRPO 的实验环境和 PPO 完全相同。

在代码实现上，除了 GRPO 与 PPO 在算法层面的不同，其余部分，包括 sandbox 代码执行、message 更新、RM 评分等方面完全相同。

GRPO 和 PPO 最显著的差异在于，GRPO 没有 Critic 预测期望 value。GRPO 在 rollout 时，会对一个 prompt 采集多个 episode，然后对 reward 在组内进行 z-score 标准化得到 advantage。Advantage 大于 0 的 episode 被认为好于平均水平，被鼓励；反之，被惩罚。Deepseek 原论文中处理的是单轮问答，而本次实验是最多 3 轮交互。为了最大程度的接近原论文实现，实验中选择把中间轮的 reward 加到最终的 final_reward 上，使用这个累加后的 final_reward 进行组内标准化。

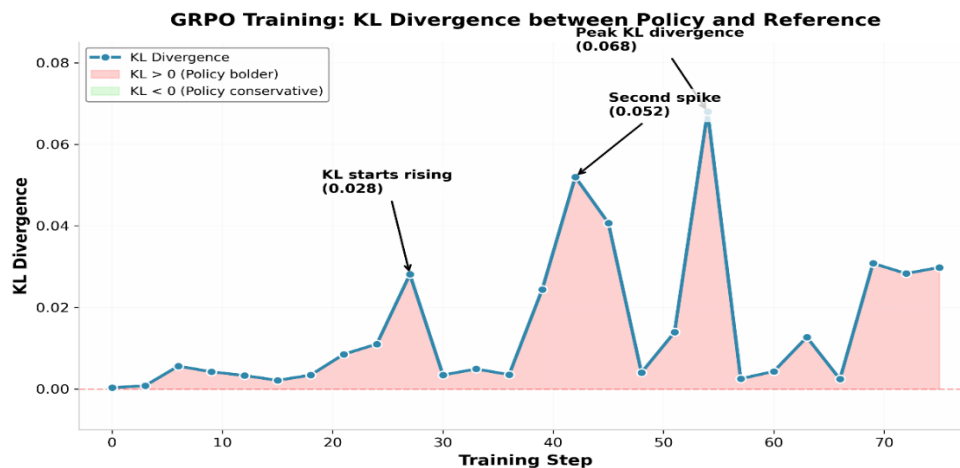
本次实验中，bs=2，group_size=4，grpo_epoch=3， $\epsilon=0.1$ ，timeout_penalty=-1.3。

2.3.2 实验结果



GRPO 和 PPO 施加了相同的 timeout_penalty (-1.3)，但 GRPO 在多轮交互方面显著保守得多——在训练中后期 timeout_rate 全部为 0，并且 average_rounds 不超过 1.5 轮。GRPO 的 loss 持续在 0 附近震荡，其中几次剧烈变化的 step 与 KL 局部峰值同步。GRPO 使用 z-score 标准化来计算组内 advantage，这可能会放大组内各 episode 的差异，导致 Policy 的 loss 波动性较大。

从下图可以看到，KL 散度始终不超过 0.07，这表明 Policy 没有大幅偏离 Reference，训练在信任域内持续进行。



注：1. 受限于 A800 的显存和训练成本，没有进行系统性调参。

2. 为了保持报告简洁，没有详细介绍代码，如果需要了解详情，请查看文件 RL_For_LLM.md。

3. 算法优劣

3.1 PPO 优势 / GRPO 劣势

1) critic 为每个 token 计算期望 value，实现计算 token-level 的 advantage，更加精细化的衡量各个 token 的贡献；而 GRPO 的 sequence-level 的 advantage 太粗糙，没法精细的区分哪些 token 更加重要。

在 PPO 中，考虑 TD err:

$$\delta_{\{i,t\}} = r_{\{i,t\}} + \gamma \cdot V_{\{i+1,t\}} - V_{\{i,t\}}$$

$r_{\{i,t\}}$ 指的是第 t 轮交互中生成第 i 个 token 得到的 reward。

GAE (Generalized Advantage Estimation) 的计算公式如下：

$$GAE_{\{i,t\}} = \delta_{\{i,t\}} + \lambda \cdot \gamma \cdot GAE_{\{i+1,t\}}$$

$$GAE_{\{last,t\}} = 0$$

对于每一轮的每一个 token，都会有自己的 advantage，产生独立的贡献。而在 GRPO 中，整个 episode 的所有 token 共享一个 advantage。

2) PPO 对于每个样本只需要收集一个 episode，而 GRPO 需要对一个样本收集多个 episode，这使得相同显存条件下，PPO 中 Policy 可以接触更加多样化的样本，这能增强 PPO 中 policy 的泛化能力。

在本次实验中，PPO 的 bs=8，而 GRPO 即使少了 Critic，依旧只能实现 bs=2、group_size=4 的实验配置。因为 PPO 中 policy 和 critic 的更新是依次进行的，所有减少了 critic 不会使得显存占用峰值减半。

3) PPO 有 entropy_loss 项，鼓励 policy 进行多样性探索，而 GRPO 的探索依赖于模型的 temperature 设置。

在本次训练中，GRPO 更倾向于一轮完成回答，而 PPO 的交互轮数波动明显。GRPO 的 timeout_penalty 和 PPO 的 timeout_penalty 相同，但 average_rounds 存在显著差异，GRPO 缺少 entropy_loss 项可能是导致这种差异的重要原因。

3.2 GRPO 优势 / PPO 劣势

1) GRPO 无需 critic，模型占用的显存比 PPO 更小。

2) GRPO 没有 critic、advantage 是 sequence-level 的，这意味着 GRPO 需要调试的超参数比 PPO 更少。

在本次实验中，GRPO 相比 PPO 减少了 GAE 计算时的两个超参数和 critic 在 lora 微调时的超参数。在系统性调参时，GRPO 需要调整的超参数显著更少。

3) GRPO 使用组内 z-score 标准化计算 sequence-level 的 advantage，在 group_size 设置合理的情况下，可以起到平滑更新的作用。policy 依据组内相对优劣信号进行学习，而 PPO 中可能出现个别样本 advantage 显著大于其他样本，主导了更新方向和幅度。

在 KL 散度代理估计的变化中，PPO 从负到正，并且出现了激增的情况；GRPO 的 KL 散度虽然存在起伏，但相比 PPO 幅度都较小。

3.3 劣势

1) PPO：在多层交互中，critic 其实很难准确估计期望 value。在本次实验后期，timeout_rate 和 kl 散度都逐渐增加，最后模型因为可能尝试了激进的使用代码导致 OOM。Critic 可能错估了此类高风险行为的期望 value，导致它错误的指导 policy 激进

的使用代码。

2) GRPO: advantage 通过组内 z-score 标准化计算得到, 如果组内的分数原本都趋近于-1, 在标准化后, 各 episode 的 advantage 区分性被严重放大, 这使得 Policy 向那个可能也很差但是比其他 episode 好一点的方向更新。更严重的是, 那个更好一点的情况可能只是随机性导致的, 比如正确的输出的<think>格式, 但是这依旧被当作好得多的情况进行学习。

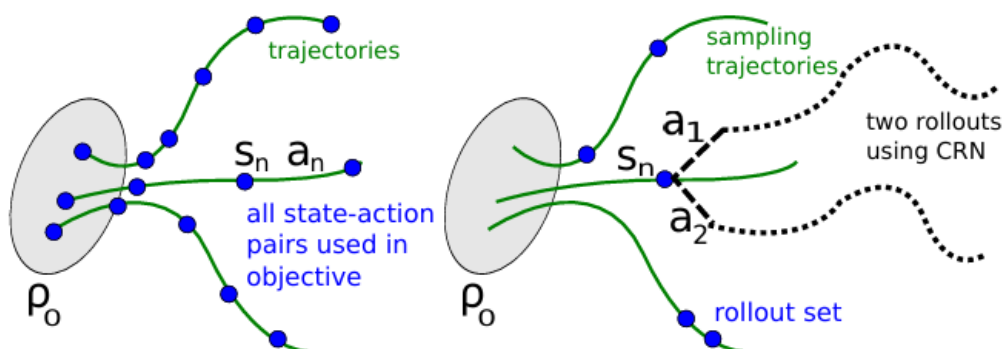
4. Rollout 方式分析

4.1 TRPO rollout 方式

在 TRPO 中提到了 single path 和 vine 两种 rollout 方式。

Single path 是先收集 $s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T$ 这样一条完整的 trajectory, 然后计算累计奖励。

Vine 首先也会收集若干段 trajectories, 然后在这些 trajectories 中选择若干 state 组成 rollout set。以 rollout set 里面的每一个 state 为起点, 如果 action space 有限且小, 就遍历每一个 action; 如果 action space 大或者无限, 就选择 K 个 action。从 (s_i, a_k) 为起点开始再收集一个短的 trajectory, 计算 $Q_\theta(s_i, a_k)$ 。



简单来说, single path 会收集一条条完整的 trajectories; vine 会从已收集到的 trajectories 的某些 states 开始, 尝试收集强制下一步采取不同 action 时的、短的 trajectories。

4.2 GRPO

GRPO 对一个 prompt 收集若干个完整的 trajectory, 然后在组内进行标准化计算 advantage, 它借鉴了 single path。

4.2.1 设计理由

基于原文，GRPO 的设计原因主要有两个：

- 1) critic 通常是和 policy 同等级规模的模型，带来了很大的计算和存储负担；
- 2) 在 LLM 领域里，通常只有最后一个 token 会被 RM 赋予 reward，这不利于训练预测每一个 token 期望 value 的 critic。

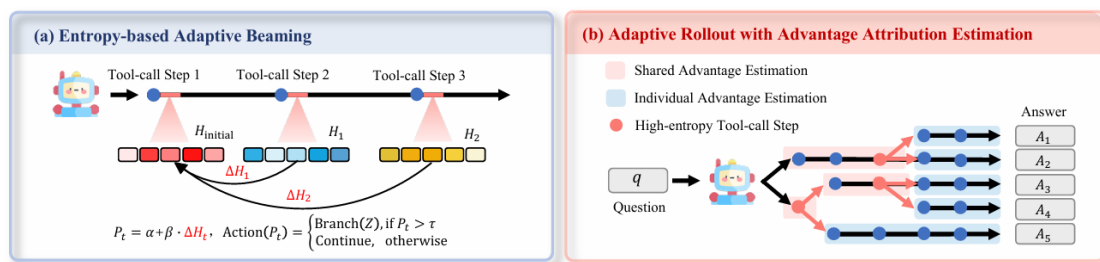
4.2.2 优点

- 1) 论文提到 RM 通常被训练为比较相同问题的不同回答，组内优势比较利用了 RM 这一特点；
- 2) RM 减少了计算和存储负担，避免了在 LLM 的 RL 中难以准确训练的 critic。

4.2.3 缺点

- 1) group_size 的大小需要合适的权重，太小会导致 advantage 方差太大，太大会导致计算量太大；
- 2) sequence-level 的 advantage 不足以帮助 policy 区分哪些 token 对正确答案的贡献更大；
- 3) GRPO 依赖组内的相对优劣信号进行学习，而 policy 的多样性探索依赖于 temperature 设置，组内各个 episode 的真实差异可能较小，学习进行缓慢。

4.3 ARPO



ARPO 把 rollout size 设定 N，先收集 M 条完整的 trajectories，然后进行 partial sampling。在已经收集到的 trajectory 上，它使用 inputs 输入后生成的 k 个 token 计算熵矩阵 $H_{initial}$ ，然后在 trajectory 每次工具调用节点处，在 response 拼接后让模型额外生成 k 个 token，使用它们计算一个熵矩阵 H_i 。计算 $\Delta H_t = \text{Normalize}(H_t - H_{initial})$ ，然后按照图中的公式决定是否进行分支采样。

在已有的 trajectory 上的 node 处使用熵比较策略决定是否分支，这借鉴了 vine。

4.3.1 设计理由

作者观察到，在接收工具调用的结果后，token entropy 会上升，这意味着模型会更加“迷茫”。特别观察到，搜索引擎返回的结果往往比 python 解释器返回的结果，使

模型更“迷茫”。作者认为，这是由于外部反馈与模型内部推理之间存在分布偏移，并且搜索引擎返回的结果比结构化的代码执行结果引入了更多的不确定性。

作者认为 PPO、GRPO 等把一条 trajectory 作为整体优化是不恰当的，因为这忽略了工具调用引入的不确定性。于是设计了 ARPO 这个利用 token 熵信号，来指导模型进行工具调用探索的机制。

4.3.2 优点

- 1) 增强了模型的不确定性探索，使得 LLM 能够更有效的识别 step-level 的工具调用行为；
- 2) 把计算复杂度从 $O(n^2)$ 降低到介于 $O(n \log n)$ 与 $O(n^2)$ 之间，因为虽然依旧是收集 n 条 trajectories，但是有些是分支探索得到的，不是一条独立的 trajectory。

4.3.3 缺点

- 1) 只适合于有外部反馈引入不确定性场景的 RL；
- 2) 需要适当的设置熵矩阵计算所需要的 token 数量 k 、基础采样概率 α 和熵采样权重 β ， M 和 N 也需要手动调整；
- 3) 相比起 PPO、GRPO 等方式，ARPO 的实现更加复杂，即需要每次计算熵矩阵，还需要分别处理 shared advantage estimation 和 individual advantage estimation。

5. Online filter

在 PPO 的实现中，我们会对 advantage 进行 batch-level 的归一化，使得最终均值为 0、方差为 1。为了将整个 batch 的 advantage 的方差控制到指定范围，一个直接的做法是对 advantage 进行缩放。

我们首先对整个 batch 减去均值 mean，确保后续缩放不改变均值。考虑到 $var(kx) = k^2 var(x)$ ，我们计算当前方差和限制边界的比值 k^2 ：

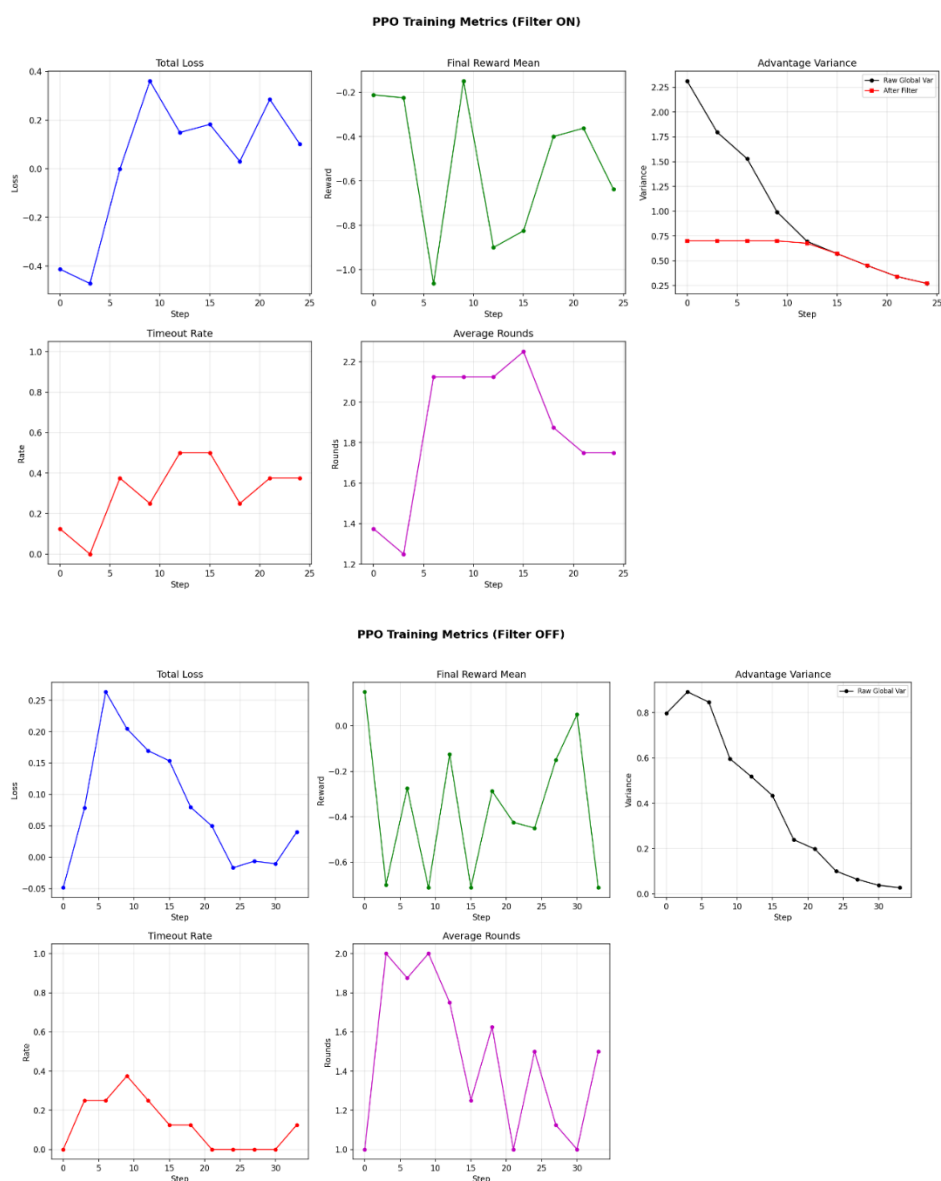
$$\begin{aligned} & \text{if } var < var_{lower}, \quad k^2 = \frac{var_{lower}}{var} \\ & \text{else if } var > var_{upper}, \quad k^2 = \frac{var_{upper}}{var} \\ & \text{else } k = 1 \end{aligned}$$

然后对 advantage 进行缩放：

$$A = A * k$$

显然: $\text{var}(kA) = k^2 \text{var}(A) \in [\text{var}_{\text{lower}}, \text{var}_{\text{upper}}]$

在最后，我们为 A 加上均值 mean，确保只改变方差。



上面是分别使用 filter 和不使用 filter 时的一次训练情况。开启 filter 时，final_reward 指标的总体波动性更弱。特别是在第 12 step 及之后的 final_reward，波动性明显弱于关闭 filter 时的情况。

我认为原因在于，开启 filter 后 advantage 被缩放来确保方差保持指定范围内——这使得学习过程稳定进行。advantage 方差过大时，会使得各个 token 对应的梯度更加混乱。由于 clip 机制限制了 ration 的范围，那些极端偏离均值的 advantage 会主导梯度更新方向，导致模型更新不稳定。advantage 方差过小时，说明此时 critic 训练得“太好”，已经没法提供区分性指导信号，导致 policy 不能从新样本中学习。模型面对新问题能够获得怎样的 final_reward，只取决于模型是否本来就擅长它，而不取决于此前的

学习过程。